Modified DSA Mini Project Documentation

58. COMPARISON OF TWO TREES — MODIFIED DSA MINI PROJECT

1. TITLE

Comparison of Two General Trees Using Recursion.

2. PROBLEM STATEMENT

Write a C++ program to construct two general trees and compare their structure and corresponding node values. The program m

3. OBJECTIVES

• Construct two general trees.
• Compare structure and corresponding node values.
• Identify the first mismatch.
• Calculate and compare node counts and heights.
• Validate inputs and release allocated memory.

4. DATA STRUCTURE

Each node stores an integer value and a vector of pointers to its children. Node 1 is the root. Child order is significant, so the firs

5. INPUT FORMAT

For each tree:
• Enter a positive number of nodes.
• Enter a value for each node ID from 1 to n.
• For each node ID in order, enter the child count followed by the child IDs.
• Node 1 is the root. Every other node must have exactly one parent and be reachable from the root.

Example for a three-node tree:

Node values: `10`, `20`, `30`
Node 1 child count: `2`; child IDs: `2 3`
Node 2 child count: `0`
Node 3 child count: `0`

6. ALGORITHM

Read and validate both trees, calculate node counts and heights, recursively compare corresponding nodes, record the first mis

7. MODIFIED FEATURES

A. Difference classification:
• Identical in structure and values.
• Structure differs only.
• Values differ only.
• Both structure and values differ.

B. First mismatch:

The program displays a path such as “root –> child 1” and the two values for a value mismatch. For a structural mismatch, it dis

C. Node count:

Displays the number of nodes in both trees and whether the counts match.

D. Height:

Displays the height of both trees. Height is measured in nodes on the longest root–to–leaf path; a single–node tree has height 1

8. SAMPLE OUTPUT (ILLUSTRATIVE)

========== TREE STATISTICS ==========
Tree 1 – Node count: 3, Height: 2
Tree 2 – Node count: 3, Height: 2
Node counts: MATCH
Heights: MATCH

========== COMPARISON RESULT ==========

Result: The trees differ in NODE VALUES only.
First difference: Value mismatch at root –> child 1: Tree 1 = 20, Tree 2 = 99.
Note: Matching node counts and heights do not prove that trees are identical.

9. TIME AND SPACE COMPLEXITY

Time:
• Tree construction: O(n) apart from input overhead.
• Node count and height calculation: O(n).
• Comparison: proportional to the corresponding nodes visited.

Space:

• Tree storage: O(n).
• Recursive call stack: O(h), where h is tree height.

10. APPLICATIONS

• Comparing hierarchical data structures.
• Comparing directory or folder structures.
• Comparing syntax trees in compilers.
• Detecting changes in organizational structures.
• Comparing nested data.

11. CONCLUSION

The modified program compares two general trees by examining structure and corresponding node values. It classifies differenc